Enterprise Desktop Support Runbook Library for Intune-Managed Windows

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

What's included . . . 4

Why it matters . . . 4

Bottom line . . . 5

Solution Overview 5

What this solution is . . . 5

The support operating model . . . 5

How it maps to an Intune-managed fleet . . . 6

How the runbooks are structured . . . 6

Requirements 7

R1. Intune role with read access to devices (technician role) . . . 7

R2. Graph read access for the reporting scripts . . . 7

R3. Remote Help (or an approved remote-control tool) . . . 8

R4. Asset and user context (who owns what) . . . 8

R5. A place to record runbook outcomes (ticketing / notes) . . . 8

Runbook Library 9

Runbook 1 — Sign-in and user profile . . . 9

Runbook 2 — OS and Windows Update . . . 10

Runbook 3 — Hardware and peripherals . . . 11

Runbook 4 — Application . . . 12

Runbook 5 — Network reachability . . . 14

Runbook 6 — Enrollment and compliance . . . 15

Cross-runbook notes . . . 16

Testing 16

How to test a runbook . . . 16

Pass/fail gates per runbook . . . 17

Script tests (read-only, run first) . . . 18

Troubleshooting 18

Symptoms that cross runbook boundaries . . . 18

Confusing single-symptom cases . . . 19

When scope changes the answer . . . 19

Tooling and script edge cases . . . 19

When nothing fits . . . 20

Best Practices 20
Consistent triage . . . 20
Evidence capture . . . 20
Escalation . . . 21
Live-environment safety . . . 21
Keeping the library healthy . . . 21
Deliverables Summary 22
Documents . . . 22
PowerShell scripts . . . 22
How the pieces fit together . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary

The problem

On a managed Windows fleet, desktop support issues get **bounced between tiers** because there is no standard way to triage them. Two technicians handed the same ticket diagnose it differently, apply different fixes, escalate at different points, and capture different (or no) evidence. The result is inconsistent resolutions, lost context on hand-off, and problems that are passed along rather than **owned end to end**.

The solution

The **Enterprise Desktop Support Runbook Library for Intune-Managed Windows** gives every technician a repeatable, click-to-implement path for the common desktop categories. It is built for an Intune + Entra + Windows environment and is deliberately white-label and vendor-neutral: the Microsoft steps are documented at exact detail, and third-party product internals are deferred to those vendors' own documentation.

Its foundation is one simple operating model applied everywhere: **intake** → **triage** → **diagnose** → **resolve/escalate** → **document.** Each runbook has a fixed anatomy — a **triage tree**, ordered **checks**, a **resolution**, an explicit **escalation gate**, and the **evidence to collect** — so any technician reaches the same correct outcome.

What's included

- **Six categorized runbooks:** sign-in/profile, OS/Windows Update, hardware/peripheral, application, network reachability, and enrollment/compliance.
- **Five PowerShell scripts:** endpoint health/context discovery, per-device state validation, a support-metrics/health report (all read-only), plus safe common-fix remediation and a target-group builder (both default to `-WhatIf`).
- **Requirements, Testing, Troubleshooting, and Best Practices** guides — with pass/fail gates per runbook and guidance for consistent triage, evidence, and escalation.

Why it matters

- **Consistency:** every technician resolves the same issue the same way.
- **Ownership:** clear escalation gates mean issues are handed off with complete evidence, not just handed off.
- **Safety in a live environment:** least-disruptive fixes first; destructive actions require approval and backup; state-changing scripts default to `-WhatIf`.
- **Reuse and cost:** a library and scripts you keep and run across the fleet — a fraction of the cost of ad-hoc consulting.

Bottom line

Give every technician a repeatable way to **triage it, resolve it, and own it** — end to end.

Solution Overview

What this solution is

This is a **runbook library** — a set of standardized, step-by-step diagnostic-to-resolution procedures for the desktop issues a Tier-1/Tier-2 technician handles on an **Intune-managed Windows fleet**. Instead of every technician improvising, the library gives each common problem category a single repeatable path: **triage tree** → **checks** → **resolution** → **escalation gate** → **evidence to collect**.

The goal is simple: any technician, at any skill level, can pick up a ticket, follow the matching runbook, and either resolve the issue the same way every other technician would, or escalate it with the right evidence already attached. Problems get **owned end to end** instead of bounced between tiers.

This library is deliberately **white-label and vendor-neutral**. It documents the Microsoft Intune, Microsoft Entra, and Windows steps at exact click-by-click detail. Where a third-party product (a specific antivirus console, a specific VPN client, a specific line-of-business app) is involved, the runbook tells you *what* to check and *when* to hand off to that vendor's own documentation, rather than guessing at another product's internals.

The support operating model

Every runbook in this library follows the same five-stage operating model. Learn it once and it applies to every category.

1. **Intake** — Capture the ticket cleanly. Who is the user, which device (Intune device name /

serial), what exactly is happening, when did it start, and how many people are affected (one user, one device, one location, or the whole fleet). Scope decides everything downstream.

1. **Triage** — Use the runbook's **triage tree** to classify the issue into a category and a

likely cause branch. Triage answers: *which runbook applies, and which branch of it?*

1. **Diagnose** — Run the runbook's **checks** in order. Each check is a read-only observation

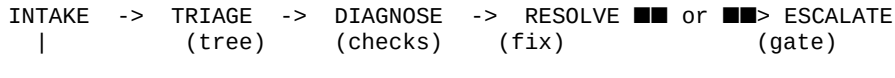
(a signal in Intune, Entra, or on the device) that confirms or rules out a cause. You stop when a check identifies the cause.

1. **Resolve or Escalate** — If the identified cause has a **resolution** in the runbook, apply

it (least-disruptive fix first). If the cause is past the runbook's **escalation gate** — it needs rights you don't have, changes that affect others, or another team's system — escalate.

1. **Document** — Record what you observed, what you did, and the outcome, using the runbook's

evidence to collect list. Good documentation is what lets the next technician (or the escalation target) continue without redoing your work.



```
+---------------------- DOCUMENT (evidence) ------------------+
```

How it maps to an Intune-managed fleet

On an Intune-managed fleet, the technician has three sources of truth, and every runbook draws on them in the same order:

- **Microsoft Intune admin center** (`https://intune.microsoft.com`) — the device's management state: enrollment, compliance, assigned policies and apps, hardware inventory, and (with the right licensing) endpoint analytics and remote actions like **Sync**, **Restart**, and **Collect diagnostics**.
- **Microsoft Entra admin center** (`https://entra.microsoft.com`) — the identity side: the user account, group memberships (which drive policy/app targeting), sign-in logs, and device join state.
- **The device itself** — reached through **Remote Help** (attended) or read-only signals; local tools such as **Settings** → **Accounts** → **Access work or school**, **Event Viewer**, **dsregcmd**, and the **Company Portal** app.

Because targeting on this fleet is **group-based**, a recurring diagnostic question across almost every runbook is: *is this device/user in the right Entra group to receive the policy or app in question?* The runbooks make that check explicit rather than assuming it.

How the runbooks are structured

Every runbook in the Runbook Library uses the same fixed anatomy so a technician always knows where to look:

- **Scope / symptoms** — the user-visible symptoms this runbook covers, and what it does *not* cover (with a pointer to the correct runbook).
- **Triage tree** — a short branching decision that narrows the issue to a probable cause path and tells you which checks to run first.
- **Checks** — an ordered list of read-only observations. Each check names exactly where to click and what a healthy vs. unhealthy result looks like.
- **Resolution** — the fix for each identified cause, least-disruptive first, written as click-to-implement steps.
- **Escalation gate** — the explicit line: what conditions mean *stop and escalate*, to whom, and what to hand over.
- **Evidence to collect** — the specific artifacts (screenshots, IDs, log names, timestamps) to attach to the ticket regardless of outcome.

The remaining documents build on this overview: **Requirements** (what a technician needs before using the library), the **Runbook Library** itself (the categorized runbooks), **Testing** (pass/fail gates that prove each runbook works), **Troubleshooting** (cross-cutting edge cases), and **Best Practices** (how to keep triage consistent, evidence complete, and escalations clean).

Requirements

Before a technician can work the runbooks in this library, a small set of prerequisites must be in place. These are **in addition to** the category-level prerequisites in `global-requirements-infrastructure.md` (Entra tenant, Intune subscription, device enrollment, supported Windows baseline, network access), which you should confirm first.

Each requirement below is written as **Description / Why / Verify / Implement if missing** so you can confirm it click-by-click and fix it if it is not there.

R1. Intune role with read access to devices (technician role)

Description: The technician's account holds an Intune role that can read device inventory, compliance, configuration, and app assignments — at minimum **Help Desk Operator**, or a custom role scoped to the same read actions plus the remote actions the team is allowed to use.

Why: Every runbook's **checks** are read operations against Intune (device state, compliance, assigned policies). Without a role that can read these, the technician is blind and every ticket becomes an escalation.

Verify: Sign in to `https://intune.microsoft.com`. Go to **Tenant administration** → **Roles** → **All roles**, open the role assigned to the technician, and confirm it includes read for **Managed devices**, **Device configurations**, and **Mobile apps**. Also open **Devices** → **All devices**, pick any device, and confirm you can open its **Overview**, **Hardware**, and **Device compliance** blades.

Implement if missing: In Intune → **Tenant administration** → **Roles** → **All roles**, select **Help Desk Operator** → **Assignments** → **Assign**. Give the assignment a name, add the technicians' Entra group under **Admin groups**, set **Scope (groups)** to the device groups they support, and choose the **Scope tags** if used. Save. Have the technician sign out and back in.

R2. Graph read access for the reporting scripts

Description: An account (or app registration) that can sign in to Microsoft Graph with the **read** scopes the bundled PowerShell scripts request: `DeviceManagementManagedDevices.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementApps.Read.All`, and `Directory.Read.All`.

Why: The Discovery, Validation, and Report scripts pull device health and context from Graph so a technician can triage across many devices at once instead of clicking each one. They are read-only and never write to the tenant.

Verify: On the technician's workstation, run `Connect-MgGraph -Scopes "DeviceManagementManagedDevices.Read.All"` and confirm sign-in succeeds and the account consents (or has admin-consented) the scope. Run `Get-MgContext` and confirm the scopes are present.

Implement if missing: Install the Microsoft Graph PowerShell SDK (`Install-Module Microsoft.Graph.Authentication -Scope CurrentUser`). If your tenant requires admin consent for these scopes, have an Intune Administrator or Global Administrator grant consent once (Entra → **Enterprise applications** → **Microsoft Graph Command Line Tools** → **Permissions**), or create a

dedicated app registration with the read scopes above and admin-consent it.

R3. Remote Help (or an approved remote-control tool)

Description: A tool that lets the technician view and, with user consent, control the device to run the on-device steps in the runbooks — Microsoft **Remote Help** on the Intune-managed fleet, or another remote-control tool the organization has approved.

Why: Several checks and resolutions require driving the local device (opening Settings, running `dsregcmd`, reading Event Viewer). Remote Help is the sanctioned, audited way to do this on an Intune fleet.

Verify: In Intune → **Tenant administration** → **Remote Help**, confirm Remote Help is **enabled**. On a test device, open the **Remote Help** app and confirm the technician can request a session and the user can grant control.

Implement if missing: In Intune → **Tenant administration** → **Remote Help**, set **Enable Remote Help** to **Yes** and configure who can provide help. Deploy the Remote Help app to the fleet (Intune → **Apps** → **Windows** → **Add** → **Microsoft Remote Help**) and assign it to the support technicians and the supported devices. Ensure the required Remote Help / Intune Suite licensing is assigned.

R4. Asset and user context (who owns what)

Description: A reliable way to map a caller to their device(s) and to the device's primary user, location, and hardware — the Intune **Primary user** field, device naming, and any asset tag or CMDB the organization keeps.

Why: Intake and triage depend on identifying the exact device. Without primary-user mapping, a technician cannot confirm they are looking at the right machine, and scope (one user vs. many) cannot be established.

Verify: In Intune → **Devices** → **All devices**, open a device and confirm the **Primary user** is populated and the **Device name** follows a recognizable convention. Confirm you can search by serial number under **Hardware**.

Implement if missing: Ensure primary user is set on enrollment (it is by default for user-driven enrollment). For shared/kiosk devices, document the exception. Adopt a device-naming convention in your enrollment profile and, if you keep a CMDB, record the Intune device name and serial there so callers can be matched to devices.

R5. A place to record runbook outcomes (ticketing / notes)

Description: A ticketing system or shared notes location where the **evidence to collect** for each runbook is stored against the ticket.

Why: The operating model's final stage is **Document**. Consistent evidence capture is what makes escalations clean and lets trends surface in the Report script.

Verify: Confirm the team's ticketing system has a field or attachment area for screenshots, device IDs, and timestamps, and that technicians know where the runbook evidence goes.

Implement if missing: Designate the evidence fields in your existing ticketing tool. If you do not have one, a shared, access-controlled location with a per-ticket note is the minimum. Standardize the fields to match the **evidence to collect** lists in the Runbook Library.

Once R1–R5 and the category-level global requirements are confirmed, the technician is ready to work the Runbook Library in the next document.

Runbook Library

This is the heart of the solution: the categorized runbooks a Tier-1/Tier-2 technician follows. Each runbook uses the fixed anatomy from the Solution Overview — **Scope / symptoms**, **Triage tree**, **Checks**, **Resolution**, **Escalation gate**, **Evidence to collect** — so you always know where to look next.

How to use this library: From the ticket, pick the runbook whose *symptoms* match. Work the *triage tree* to land on a branch, run the *checks* in order until one identifies the cause, apply the matching *resolution*, or stop at the *escalation gate*. Attach the *evidence* regardless of outcome. Always prefer the least-disruptive fix first (Sync before Restart, Restart before re-enroll, targeted change before wipe).

Two conventions used throughout:

- **Intune** = Microsoft Intune admin center, `https://intune.microsoft.com`.
- **Entra** = Microsoft Entra admin center, `https://entra.microsoft.com`.

Runbook 1 — Sign-in and user profile

Scope / symptoms

User cannot sign in to Windows or to work resources; "your account has been locked / needs attention"; wrong or temporary profile ("We can't sign you in with this profile"); repeated password/MFA prompts; SSO to Microsoft 365 apps fails. **Not covered:** the device itself is unenrolled or non-compliant (use Runbook 6); an app opens but errors (use Runbook 4).

Triage tree

```
Can the user sign in at https://portal.office.com from ANOTHER device?
■■ No  → identity-side issue (account, password, MFA, conditional access) → Checks C1–C3
■■ Yes → device/profile-side issue on THIS device                          → Checks C4–C5
```

Checks

- **C1 — Account state (Entra):** Entra → **Users** → the user → confirm **Account enabled** = Yes,

not blocked, and sign-in not marked risky. Unhealthy: account disabled or blocked.

- **C2 — Sign-in logs (Entra):** Entra → **Users** → the user → **Sign-in logs**. Read the most

recent failure's **Failure reason** and the **Conditional Access** tab. Unhealthy: a Conditional Access policy is blocking, or MFA is failing.

- **C3 — MFA methods (Entra):** Entra → **Users** → the user → **Authentication methods**. Unhealthy:

no usable method, or the user lost their device.

- **C4 — Profile type (device, via Remote Help): Settings** → **Accounts** → **Your info** and

Event Viewer → **Applications and Services Logs** → **Microsoft** → **Windows** → **User Profile Service**. Unhealthy: "temporary profile" event (User Profile Service event ID 1511/1515).

- **C5 — Cached credentials / join state (device):** run `dsregcmd /status`. Confirm

`AzureAdJoined : YES` and `SSO State` shows a valid PRT (`AzureAdPrt : YES`). Unhealthy: `AzureAdPrt : NO` → SSO/token problem.

Resolution

- **Locked/disabled account (C1):** if within your rights, re-enable in Entra → **Users** → user →

Account enabled. If it was disabled deliberately, escalate.

- **MFA lost/broken (C3):** if permitted, have the user re-register a method, or request a

Temporary Access Pass from an identity admin. Do **not** disable MFA.

- **Temporary profile (C4):** confirm free disk space, then reboot. If it persists, the profile is

corrupt — escalate (profile rebuild affects user data; see gate).

- **Missing PRT (C5):** run `dsregcmd /status`, have the user lock/unlock or sign out and back in

to re-acquire the PRT; confirm the device time/date is correct (token errors are often clock skew).

Escalation gate

Escalate to the **identity/security team** when: a Conditional Access policy must be changed (C2); an account was disabled on purpose; MFA reset requires elevated identity rights; or a profile is corrupt and must be rebuilt (data-loss risk — never delete a profile without the user's data backed up and owner approval).

Evidence to collect

Screenshot of the sign-in log failure reason and Conditional Access result; `dsregcmd /status` output; User Profile Service event IDs with timestamps; the exact error text shown to the user.

Runbook 2 — OS and Windows Update

Scope / symptoms

Windows won't boot or is slow/unstable; update stuck, failing, or looping; a feature update won't install; blue-screen (bugcheck). **Not covered:** a single app crashing (Runbook 4); hardware faults (Runbook 3).

Triage tree

```
Does the device boot to the desktop?
■■ No  → boot/stability issue                → Checks C1–C2 (then likely escalate)
■■ Yes → is it an UPDATE problem or PERF/stability?
         ■■ Update    → Checks C3–C4
         ■■ Perf      → Check C5
```

Checks

- **C1 — Recent change (Intune):** Intune → **Devices** → device → **Recent activity** and

Managed apps — did a policy, feature update, or app deploy just before the failure?

- **C2 — Bugcheck history (device): Event Viewer** → **Windows Logs** → **System**, filter Source

`BugCheck`. Note the stop code. Recurring identical stop code → likely driver/hardware (cross-check Runbook 3).

- **C3 — Update ring / status (Intune):** Intune → **Devices** → **Windows** → **Update rings** (or

Feature updates) → confirm the device's ring and **deferral/pause** settings; open the device's **Windows 10 and later updates** report for the failing update and its error code.

- **C4 — Update error (device): Settings** → **Windows Update** → **Update history**; note the

`0x…` error code. Check free disk space (feature updates need headroom).

- **C5 — Performance (Intune):** if endpoint analytics is licensed, Intune →

Reports → **Endpoint analytics** → **Startup performance** for this device; otherwise Task Manager / Resource Monitor via Remote Help.

Resolution

- **Stuck/failed quality update:** trigger **Sync** (Intune → device → **Sync**), then on the

device run `Settings → Windows Update → Retry`. If it fails again, note the error code for the gate.

- **Low disk blocking update (C4):** run **Storage Sense** / **Disk Cleanup** (incl. Windows

Update cleanup) via Remote Help, then retry.

- **Paused/deferred by policy (C3):** confirm the ring is intended; if the device is wrongly

excluded from an update ring, fix group membership (see Runbook 6 / Set-ScopeTargeting) rather than forcing updates locally.

- **Performance (C5):** identify the top CPU/disk/memory consumer; if it's a known app, apply

Runbook 4; if startup is slow, review startup apps.

Escalation gate

Escalate to the **desktop engineering / platform team** when: the device does not boot (C1/C2); a recurring bugcheck points to driver/firmware; a feature update fails with the same error after disk and retry are cleared; or an update-ring policy needs to change. **Never** re-image or reset a device without owner approval and confirmed data backup.

Evidence to collect

Windows Update error code(s); relevant System-log bugcheck stop codes with timestamps; free-disk figure; the device's update ring name; endpoint-analytics startup score if available.

Runbook 3 — Hardware and peripherals

Scope / symptoms

Device won't power on / battery/charging issue; display, keyboard, touchpad, camera, audio, or docking-station peripheral not working; external monitor not detected; recurring device-driver errors. **Not covered:** the peripheral is fine but the app can't use it (Runbook 4); network hardware/reachability (Runbook 5).

Triage tree

```
Is the failing part INTERNAL (screen, keyboard, battery) or a PERIPHERAL (monitor, dock, USB, headset)?
■■ Internal   → likely physical → Checks C1–C2 → often escalate to hardware/warranty
■■ Peripheral → Checks C2–C4 (driver, connection, another port/device)
```

Checks

- **C1 — Warranty / model context (Intune):** Intune → **Devices** → device → **Hardware** — note model, serial, and (if recorded) warranty. This tells you the escalation path for physical repair.
- **C2 — Device Manager (device, via Remote Help):** open **Device Manager**; look for a yellow bang / "Unknown device" / disabled device. Note the device name and any error code (e.g. Code 43, Code 10).
- **C3 — Isolate the peripheral:** try another port, another cable, and the same peripheral on a second machine (or a different peripheral on this machine). This separates "device fault" from "peripheral fault."
- **C4 — Driver source (Intune):** confirm whether the driver is delivered by an Intune app/driver policy or Windows Update; check **Update history** for a recent driver change that lines up with the failure.

Resolution

- **Peripheral driver (C2/C4):** via Remote Help, in Device Manager **Uninstall device** (leave "delete driver" unchecked unless the driver is known-bad) and **Scan for hardware changes**, or let Windows Update re-provision it; reconnect.
- **Docking/monitor not detected (C3):** reseat the dock, update the dock firmware per the *dock vendor's documentation*, try the port directly; confirm the correct display input.
- **Peripheral confirmed faulty (C3):** arrange replacement of the peripheral.
- **Internal fault confirmed (C1/C3):** escalate to hardware/warranty (see gate).

Escalation gate

Escalate to **hardware / warranty (or the OEM per the device vendor's documentation)** when an internal component is confirmed faulty, the device won't power on, or a repair is needed. Escalate to **desktop engineering** if a bad driver is being delivered fleet-wide by policy.

Evidence to collect

Model + serial from Hardware; Device Manager error code and device name; isolation results (which port/cable/second machine was tried and the outcome); any recent driver version change.

Runbook 4 — Application

Scope / symptoms

An app won't install, won't open, crashes, or errors on launch; an Intune-deployed app is missing; a line-of-business app misbehaves. **Not covered:** the app is fine but sign-in to it fails (Runbook 1); the app can't reach its server (Runbook 5).

Triage tree

```
Is the app DEPLOYED by Intune, or user/manually installed?
■■ Intune-deployed → is it INSTALLED on the device?
■   ■■ No  → deployment/targeting issue → Checks C1–C2
■   ■■ Yes → runtime issue              → Checks C3–C4
■■ User-installed  → runtime/support issue → Checks C3–C4 (+ vendor docs)
```

Checks

• **C1 — App assignment (Intune):** Intune → **Apps** → **All apps** → the app → **Device install

status / User install status**; find the device/user and read the status (Installed, Failed, Not applicable) and any error code.

• **C2 — Targeting (Entra/Intune):** confirm the device or user is in the Entra group the app is

assigned to (App → **Properties** → **Assignments**). "Not applicable" almost always means the target isn't in the group.

• **C3 — App crash (device): Event Viewer** → **Windows Logs** → **Application**, filter Source

`Application Error` / `.NET Runtime`; note the faulting module and event time.

• **C4 — Version / dependency (device):** confirm installed version (**Settings → Apps →

Installed apps**) against the expected version; confirm prerequisites/runtimes are present.

Resolution

• **Failed/missing deploy (C1):** trigger **Sync** on the device; if still failing, read the

install error code; if targeting is wrong (C2), fix group membership (Runbook 6 / Set-ScopeTargeting) — do not sideload around the deployment.

• **Crashing app (C3):** repair or reinstall the app (via Company Portal for Intune apps),

update to the supported version, install the missing runtime/dependency (C4).

• **Vendor-specific fault:** if the app's own logs point to a product-internal error, hand off to

that application vendor's documentation/support with the evidence attached.

Escalation gate

Escalate to the **app owner / packaging team** when an Intune app install fails fleet-wide, the package or detection rule is wrong, or a licensing/entitlement issue is involved. Escalate to the **application vendor** for product-internal faults beyond reinstall/repair.

Evidence to collect

App name and version; Intune install status + error code; whether the target is in the assigned group; Application-log faulting module/event with timestamp; steps already tried (sync, reinstall, repair).

Runbook 5 — Network reachability

Scope / symptoms

No internet or intranet; can't reach a specific server, share, or SaaS site; VPN won't connect; Wi-Fi drops; "you're offline"; slow connectivity. **Not covered:** the network is fine but sign-in fails (Runbook 1); an app-internal error (Runbook 4).

Triage tree

```
Is EVERYTHING unreachable, or just ONE destination?
■■ Everything → local network/adapter/Wi-Fi → Checks C1–C2
■■ One target → name resolution / route / that service → Checks C3–C4
```

Checks

• **C1 — Adapter / link (device, via Remote Help): Settings** → **Network & internet**; confirm

connected SSID/Ethernet, and run `ipconfig /all` — valid IP (not 169.254.x.x), gateway, DNS.

• **C2 — Basic reachability:** `ping` the gateway, then `ping 8.8.8.8` (raw IP) — isolates

local vs. upstream.

• **C3 — Name resolution:** `nslookup <the-failing-hostname>` — does it resolve? Compare a

working host. DNS failure vs. reachable-but-refused are different problems.

• **C4 — Path / VPN / proxy:** `Test-NetConnection <host> -Port <port>` for the specific service;

if VPN, check the VPN client state and the VPN profile in Intune (**Devices** → **Configuration** → VPN profile → assignment). Confirm proxy settings aren't misdirecting traffic.

Resolution

• **No IP / 169.254 (C1):** reconnect Wi-Fi or reseat Ethernet; `ipconfig /release` + `/renew`; forget/rejoin the Wi-Fi profile if corrupt.

• **DNS (C3):** `ipconfig /flushdns`; confirm the device is using the expected DNS servers (from DHCP or policy), not a stale/manual entry.

• **VPN (C4):** reconnect; confirm credentials/cert; confirm the VPN profile is assigned to the device's group (Runbook 6 if not); update the VPN client per the *VPN vendor's documentation*.

• **One SaaS/site only:** confirm it's reachable from another network — if it's down for everyone, it's a service outage, not this device.

Escalation gate

Escalate to the **network team** when the fault is upstream (gateway/DNS/firewall/proxy serving the whole location), a VPN gateway is down, or a firewall rule must change. Escalate to the **service owner** for a confirmed SaaS/service outage. Do not change firewall or DNS infrastructure from the endpoint.

Evidence to collect

`ipconfig /all` output; ping/`Test-NetConnection` results (which target, which port, pass/fail); `nslookup` result for the failing host; VPN profile name and assignment; whether the target is reachable from another network.

Runbook 6 — Enrollment and compliance

Scope / symptoms

Device shows non-compliant; "your device needs attention" / blocked from resources by Conditional Access; device missing from Intune or not receiving policies/apps; enrollment failed; the device isn't in the group it should be. **Not covered:** the device is enrolled and compliant but a single app failed (Runbook 4).

Triage tree

```
Is the device ENROLLED and visible in Intune?
■■ No  → enrollment/join problem → Checks C1–C2
■■ Yes → is it COMPLIANT?
         ■■ No       → which setting fails? → Check C3
         ■■ Yes      → not getting a policy/app → Check C4 (targeting)
```

Checks

• **C1 — Presence (Intune):** Intune → **Devices** → **All devices** — is the device there? Note

Managed by, **Ownership**, **Last check-in**. Stale last check-in → the device isn't syncing.

• **C2 — Join state (device):** `dsregcmd /status` — `AzureAdJoined : YES`,

`EnterpriseJoined`/`DomainJoined` as expected, and `MDMUrl` present (enrolled). Unhealthy: joined but not MDM-enrolled.

• **C3 — Compliance detail (Intune):** Intune → **Devices** → device → **Device compliance** —

open the failing policy and read exactly which setting is non-compliant (e.g. BitLocker, Defender, minimum OS, password).

• **C4 — Targeting (Entra/Intune):** confirm the device (or its primary user) is in the Entra

group that the policy/app is assigned to. Missing membership = missing policy.

Checks → Resolution

• **Not syncing (C1):** Intune → device → **Sync**; on the device open **Settings → Accounts →

Access work or school → **the account** → Info → Sync**. Confirm network (Runbook 5).

• **Joined but not enrolled (C2):** confirm auto-enrollment; have the user

Access work or school → **Connect / re-connect** the work account; if it fails, note the error.

• **Compliance setting (C3):** remediate the specific setting — e.g. turn on BitLocker/Defender,

set a compliant password, install the pending update (Runbook 2). Re-run compliance evaluation (**Sync**).

• **Wrong group (C4):** add the device/user to the correct Entra group (see

DESKSUP-Set-ScopeTargeting.ps1, default `-WhatIf`), or request the change if you lack rights.

Escalation gate

Escalate to **endpoint engineering** when: enrollment repeatedly fails with the same error; a compliance policy setting can't be remediated on the endpoint (e.g. hardware doesn't support a required feature); the device must be retired/re-enrolled/wiped (owner approval + data backup required); or a Conditional Access block requires an identity/security decision (also Runbook 1).

Evidence to collect

`dsregcmd /status` output; device Overview (Managed by / Ownership / Last check-in); the exact non-compliant setting and its policy name; the group(s) the device is/should be in; enrollment error text if any.

Cross-runbook notes

- **Least-disruptive first, always:** Sync → local retry → targeted config change → reinstall →

(only with approval) re-enroll/reset. Never jump to a destructive action to save a step.

- **Scope before cause:** one user, one device, one location, or the whole fleet — establish this

in intake; a fleet-wide symptom almost always means policy/targeting or a service outage, not the single device in front of you.

- **The scripts support the runbooks:** use **DESKSUP-Discovery** and **DESKSUP-Validation** during

Checks to read device state at scale, **DESKSUP-Report** to see trends for the *Document* stage, **DESKSUP-Remediation** for the safe common fixes above (default `-WhatIf`), and **DESKSUP-Set-ScopeTargeting** for the group-membership fixes (default `-WhatIf`).

Testing

Testing here means proving that each runbook actually leads a technician from symptom to a correct outcome, and that the bundled scripts read what they claim to read. Each runbook has an explicit **pass/fail gate**: the runbook "passes" when a technician following it reaches the right resolution (or the right escalation) using only the runbook's checks and evidence.

Run these on a **pilot device** (from the pilot Entra security group, G5) before relying on the library across the fleet.

How to test a runbook

1. Reproduce or select a real ticket that matches the runbook's **Scope / symptoms**.
2. Follow the **triage tree** — confirm it lands you on the correct branch.
3. Run the **checks** in order — confirm each one is observable exactly where the runbook says.
4. Apply the **resolution** (or reach the **escalation gate**) — confirm the outcome is correct.
5. Confirm every item in **evidence to collect** was actually capturable.

A runbook **fails** the gate if any check can't be observed as written, the triage tree misroutes, the resolution doesn't resolve (or escalates when it shouldn't), or required evidence can't be captured. Fix the runbook, then re-test.

Pass/fail gates per runbook

Runbook 1 — Sign-in / profile

- **PASS:** Triage correctly separates an identity-side failure (reproducible on a second device)

from a device/profile failure; `dsregcmd /status` PRT state is readable; the sign-in log failure reason is found; resolution restores sign-in **or** a clean identity/profile escalation is raised with evidence.

- **FAIL:** Technician can't tell identity from profile; no PRT/sign-in-log evidence captured;

MFA/CA change attempted on the endpoint instead of escalated.

Runbook 2 — OS / Windows Update

- **PASS:** Update error code and update ring are identified; disk/retry path is exercised;

boot/bugcheck cases route to escalation; a stuck update is cleared **or** escalated with the error code and ring name recorded.

- **FAIL:** No error code captured; a non-booting device is treated as a local retry; a re-image is

attempted without approval/backup.

Runbook 3 — Hardware / peripheral

- **PASS:** Model/serial captured; Device Manager error code found; the peripheral is isolated

(other port/cable/second machine); resolution fixes the driver/peripheral **or** a confirmed physical fault escalates to warranty with evidence.

- **FAIL:** No isolation performed; internal fault treated as a driver reinstall loop; no

model/serial recorded for warranty.

Runbook 4 — Application

- **PASS:** Install status and targeting are checked; deploy-vs-runtime is correctly distinguished;

a targeting miss is fixed via group membership (not sideloading); a crash is resolved by repair/reinstall/dependency **or** escalated to the app owner/vendor with the faulting module recorded.

- **FAIL:** App sideloaded to bypass a broken deployment; no install error code or faulting module

captured.

Runbook 5 — Network reachability

- **PASS:** "Everything vs. one target" is correctly triaged; `ipconfig /all`, ping-by-IP, and

`nslookup`/`Test-NetConnection` evidence captured; local fixes (renew/flushdns/rejoin) applied; upstream faults escalated to the network team.

- **FAIL:** DNS vs. routing not distinguished; endpoint attempts to change firewall/DNS

infrastructure; a service outage treated as a device fault.

Runbook 6 — Enrollment / compliance

- **PASS:** Presence and join state (`dsregcmd`) confirmed; the exact non-compliant setting is

identified by name; targeting/group membership verified; setting remediated and re-evaluated **or** escalated; no wipe/re-enroll without approval + backup.

- **FAIL:** "Non-compliant" reported without naming the failing setting; device wiped/re-enrolled without approval; targeting not checked when a policy is missing.

Script tests (read-only, run first)

- **DESKSUP-Discovery.ps1** — PASS when it connects to Graph with read scopes and returns endpoint health/context for the target device(s) without writing anything. FAIL if it errors on scopes or attempts a write.
- **DESKSUP-Validation.ps1** — PASS when the per-device state it reports (enrollment, compliance, key policies) matches what the Intune portal shows for a pilot device.
- **DESKSUP-Report.ps1** — PASS when the support-metrics/health summary runs read-only and its figures reconcile with the portal for the pilot scope.
- **DESKSUP-Remediation.ps1** — PASS when run with the default `-WhatIf`: it prints the intended safe fix and makes **no** change. Only after WhatIf review should it be run for real on a pilot device.
- **DESKSUP-Set-ScopeTargeting.ps1** — PASS when run with the default `-WhatIf`: it prints the intended group membership change and makes **no** change until explicitly confirmed on the pilot group.

Overall gate to leave pilot: every runbook passes on the pilot device, all five scripts pass their read-only/`-WhatIf` tests, and the evidence lists are all capturable in your ticketing tool.

Troubleshooting

The Runbook Library covers the common single-category cases. This guide handles the **cross-cutting edge cases** — tickets that span two runbooks, symptoms that mimic one cause but have another, and the failure modes of the tooling itself. Use it when a ticket doesn't sit cleanly inside one runbook.

Symptoms that cross runbook boundaries

- **"Can't open the app" that is really sign-in.** An app launches but shows a login/authentication error. This is **Runbook 1 (sign-in)**, not Runbook 4. Tell: the app opens and its UI works, but it can't authenticate; check the sign-in log and PRT (`dsregcmd /status`) before touching the app.
- **"Can't open the app" that is really network.** The app opens but "can't reach server." This is **Runbook 5 (network)**. Tell: other apps to the same server also fail; `Test-NetConnection` to the app's host/port fails.
- **"Non-compliant device" that is really an update.** Compliance fails on "minimum OS version" or a missing update. Start in **Runbook 6** to identify the failing setting, then switch to **Runbook 2** to actually get the update installed, then re-evaluate compliance.

- **"Blocked from resources" — Conditional Access vs. compliance.** Being blocked can come from a

non-compliant device (Runbook 6) *or* a Conditional Access policy (Runbook 1). The sign-in log's Conditional Access tab names which policy applied; that decides the runbook.

- **Peripheral works in one app but not another.** The hardware and driver are fine (Runbook 3

isolation passes), so it's an **app** problem — the app lacks permission or the wrong device is selected in the app. Move to **Runbook 4** (and check Windows privacy settings for camera/mic).

Confusing single-symptom cases

- **Repeated password/MFA prompts.** Could be clock skew (fix device time — token validation fails

when the clock is wrong), a broken PRT (`dsregcmd /status` → `AzureAdPrt : NO`), or a Conditional Access session policy. Check time first (cheapest), then PRT, then sign-in logs.

- **169.254.x.x address.** The adapter has no DHCP lease — treat as **no connectivity** (Runbook 5

C1), not a DNS or app issue, even if the user reports "a website won't load."

- **Update stuck at the same percentage.** Often low disk or a pending reboot, not a corrupt update.

Clear disk and reboot before assuming corruption or escalating.

- **Device "missing" from Intune.** Confirm you're searching the right identifier (device name vs.

serial vs. primary user) before concluding it's unenrolled — it may simply be named differently (Runbook 6 C1, and see Requirement R4 asset context).

When scope changes the answer

- **One device vs. many.** If several devices show the same symptom at once, stop working the single

device: it's almost certainly **policy/targeting** (wrong or missing Entra group assignment — use DESKSUP-Set-ScopeTargeting to inspect) or a **service outage** (Runbook 5 service-owner escalation). Working each device individually will not fix a fleet-wide cause.

- **New device vs. existing.** A brand-new enrollment failing differently from established devices

usually points to the enrollment profile or Autopilot/join step (Runbook 6 escalation), not the everyday runbook paths.

Tooling and script edge cases

- `Connect-MgGraph` **fails or lacks scopes.** Re-run with the explicit `-Scopes` the script header

lists; if consent is required, an admin must grant it once (Requirement R2). `Get-MgContext` confirms the active scopes.

- **A script returns fewer devices than expected.** The signed-in account's Intune **scope (groups)**

may limit what it can read (Requirement R1). Confirm the technician's role scope includes the target devices.

- **Remediation/Set-ScopeTargeting "did nothing."** By design — both default to `-WhatIf` and only

print the planned change. Re-run without `-WhatIf` (on a pilot first) once you've reviewed the plan.

- **Portal and script disagree.** The portal can lag; trigger **Sync** on the device and re-run the read-only Validation script. Persistent disagreement is itself evidence — attach both and escalate.
- **Remote Help won't connect.** Confirm Remote Help is enabled and assigned (Requirement R3), the user is present to consent, and the device has connectivity (Runbook 5) before assuming a Remote Help fault.

When nothing fits

If a ticket matches no runbook and no edge case above: capture the standard evidence (device name/serial, exact symptom, scope, timestamps, any error codes), run DESKSUP-Discovery/Validation for state, and escalate to endpoint engineering with that package. An unclassifiable-but-recurring ticket is a signal to add a new runbook — feed it back into the library (see Best Practices).

Best Practices

These practices keep the runbook library working as intended: **consistent triage**, **complete evidence capture**, and **clean escalation**. They apply across every runbook.

Consistent triage

- **Scope before cause.** In intake, always establish whether the issue affects one user, one device, one location, or the whole fleet. A fleet-wide symptom is almost always policy/targeting or a service outage — chasing it on a single device wastes time.
- **Always start at the triage tree.** Don't skip to a fix you "have a feeling about." The tree exists so two technicians reach the same branch from the same symptoms.
- **Run checks in order and stop at the cause.** Checks are ordered cheapest/least-disruptive first. When a check identifies the cause, stop — you don't need to run the rest.
- **One runbook at a time, with clear hand-offs.** If a check reveals the real issue belongs to another category (e.g. an "app" problem is really a sign-in problem), switch to that runbook and note why.
- **Least-disruptive fix first.** Sync → local retry → targeted config/group change → reinstall → (only with approval + backup) re-enroll/reset. Never jump straight to a destructive action.

Evidence capture

- **Capture as you go, not at the end.** Screenshot the failing signal at the moment you see it — sign-in log reason, compliance setting, error code, `dsregcmd` output.
- **Use the runbook's evidence list as a checklist.** Every runbook ends with an explicit *evidence to collect* list; treat it as the definition of "documented."
- **Record IDs and timestamps, not just descriptions.** Intune device name/serial, event IDs,

`0x…` error codes, and exact times let the next person (or an escalation target) continue without redoing your diagnosis.

- **Prefer read-only tooling for capture.** Use DESKSUP-Discovery/Validation/Report and the Intune portal to gather state; they don't change anything and are safe to run repeatedly.
- **Never capture secrets.** Don't screenshot passwords, tokens, TAPs, or full BitLocker keys into a ticket.

Escalation

- **Escalate at the gate, not before or after.** Each runbook names the exact conditions that mean *stop and hand off*. Escalating earlier drops ownership; escalating later delays the user.
- **Escalate with a complete package.** Hand over the collected evidence, what you already tried, and the specific decision or right the target needs to supply. A good escalation is one the receiver can act on without a follow-up question.
- **Route to the right team.** Identity/security for accounts, MFA, and Conditional Access; desktop/endpoint engineering for OS, drivers-by-policy, enrollment, and re-image; network team for upstream connectivity; app owner/vendor for packaging or product-internal faults; hardware/warranty for physical repair.
- **Own the ticket across the hand-off.** Escalation is a transfer of *action*, not of *responsibility for the user's experience* — track it to resolution.

Live-environment safety

- **Default to `-WhatIf`.** The remediation and scope-targeting scripts ship WhatIf-first; review the planned change before removing it, and run destructive-adjacent actions on a **pilot** device or group first.
- **Confirm data backup before anything that touches user data.** Profile rebuilds, re-enrollment, resets, and re-imaging require owner approval and confirmed backup — no exceptions.
- **Change targeting, not the endpoint, to fix "missing policy/app."** If a device isn't getting a policy or app, fix its Entra group membership rather than sideloading or forcing settings locally.

Keeping the library healthy

- **Feed corrections back.** When a runbook step is wrong (a menu moved, an object renamed, a check no longer shows what's described), update the runbook so the next technician benefits. Consistency is only maintained if the library stays current.
- **Watch the trend.** Use DESKSUP-Report over time — a rising category is a signal to add or refine a runbook, not just to work more tickets.

Deliverables Summary

This solution is a self-contained **runbook library** plus supporting scripts. Everything below is yours to keep and reuse across the fleet.

Documents

- **Solution Overview** (`01-solution-overview.md`) — the support operating model (intake → triage → diagnose → resolve/escalate → document), how it maps to an Intune-managed fleet, and the fixed anatomy every runbook uses.
- **Requirements Guide** (`02-requirements.md`) — the technician prerequisites (Intune read role, Graph read access, Remote Help/remote tooling, asset/user context, a place to record outcomes), each with Description / Why / Verify / Implement-if-missing, plus the category-level global requirements.
- **Runbook Library** (`03-implementation.md`) — the six categorized runbooks — sign-in/profile, OS/Windows Update, hardware/peripheral, application, network reachability, and enrollment/compliance — each with triage tree, checks, resolution, escalation gate, and evidence to collect.
- **Testing Guide** (`05-testing.md`) — pass/fail gates per runbook and per script, run on a pilot device before fleet use.
- **Troubleshooting Guide** (`06-troubleshooting.md`) — cross-cutting edge cases that span runbooks.
- **Best Practices** (`07-best-practices.md`) — consistent triage, evidence capture, and clean escalation.
- **Global Requirements — Infrastructure** (`global-requirements-infrastructure.md`) — the category-level prerequisites shared by all Infrastructure solutions.

PowerShell scripts

All scripts read from Microsoft Graph with read-only scopes; the two that can change state default to `-WhatIf`.

- **DESKSUP-Discovery.ps1** — endpoint health/context discovery across target devices (read-only).
- **DESKSUP-Validation.ps1** — per-device state validation (enrollment, compliance, key policies) (read-only).
- **DESKSUP-Report.ps1** — support-metrics / fleet-health summary report (read-only).
- **DESKSUP-Remediation.ps1** — safe common fixes (sync, cache/flush, targeted retries); defaults to `-WhatIf`.
- **DESKSUP-Set-ScopeTargeting.ps1** — build/adjust pilot and support target groups; defaults to `-WhatIf`.

How the pieces fit together

Read the **Overview** to learn the operating model, confirm the **Requirements**, then work tickets from the **Runbook Library**. Use **Discovery/Validation/Report** during the *Checks* and *Document* stages, **Remediation** for the safe fixes, and **Set-ScopeTargeting** to correct group-based targeting. Prove everything on a pilot device using the **Testing** gates, lean on **Best Practices** for consistency, and reach for **Troubleshooting** when a ticket doesn't fit a single runbook.

Outcome

Every technician resolves desktop issues the same repeatable way and owns them end to end — a runbook library and scripts you keep and reuse across the fleet, at a fraction of the cost of ad-hoc consulting.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DESKSUP-Discovery.ps1

Purpose: Discovers endpoint health and support context across the Intune-managed fleet so a technician can triage at scale instead of clicking each device. Pulls managed-device inventory, ownership, compliance state, last check-in, OS build, and primary user from Microsoft Graph. Read-only: it never changes the tenant. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DESKSUP-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\DESKSUP-Discovery.ps1

DESKSUP-Remediation.ps1

Purpose: Applies the safe, least-disruptive common fixes the runbooks call for: trigger an Intune device Sync, re-evaluate compliance, and (optionally) send a Restart. It reads device state from Microsoft Graph, then issues only the low-risk Intune device action requested. It defaults to -WhatIf and makes no change until you re-run with -WhatIf:$false. It never wipes, resets, retires, or deletes anything. HOW TO RUN Preview (no change, default): powershell -ExecutionPolicy Bypass -File .\DESKSUP-Remediation.ps1 -DeviceName device-name-or-serial -Action Sync Apply after reviewing: powershell -ExecutionPolicy Bypass -File .\DESKSUP-Remediation.ps1 -DeviceName device-name-or-serial -Action Sync -WhatIf:$false

How to run: Preview (no change, default): powershell -ExecutionPolicy Bypass -File .\DESKSUP-Remediation.ps1 -DeviceName device-name-or-serial -Action Sync Apply after reviewing: powershell -ExecutionPolicy Bypass -File .\DESKSUP-Remediation.ps1 -DeviceName device-name-or-serial -Action Sync -WhatIf:$false

DESKSUP-Report.ps1

Purpose: Produces a support-metrics and fleet-health summary a lead can use to spot trends: counts by compliance state, stale (not-checked-in) devices, OS-version spread, and ownership mix, drawn from Microsoft Graph. Read-only: it never changes the tenant. Use it during the Document stage and to decide whether a rising category needs a new or refined runbook. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DESKSUP-Report.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\DESKSUP-Report.ps1

DESKSUP-Set-ScopeTargeting.ps1

Purpose: Fixes the group-based targeting the runbooks rely on: ensures a pilot/support Entra security group exists and adds the specified devices (or their primary users) to it, so a device starts receiving the policy or app it was missing. It reads current membership from Microsoft Graph and only adds members you name. It defaults to -WhatIf and makes no change until you re-run with -WhatIf:$false. It never removes members or deletes groups. HOW TO RUN Preview (no change, default): powershell -ExecutionPolicy Bypass -File .\DESKSUP-Set-ScopeTargeting.ps1 -GroupName pilot-group-name -DeviceName device-name-or-serial Apply after reviewing: powershell -ExecutionPolicy Bypass -File .\DESKSUP-Set-ScopeTargeting.ps1 -GroupName pilot-group-name -DeviceName device-name-or-serial -WhatIf:$false

How to run: Preview (no change, default): powershell -ExecutionPolicy Bypass -File .\DESKSUP-Set-ScopeTargeting.ps1 -GroupName pilot-group-name -DeviceName device-name-or-serial Apply after reviewing: powershell -ExecutionPolicy Bypass -File .\DESKSUP-Set-ScopeTargeting.ps1 -GroupName pilot-group-name -DeviceName device-name-or-serial -WhatIf:$false

DESKSUP-Validation.ps1

Purpose: Validates the support-relevant state of a single device so a technician can confirm what a runbook check expects. Reports enrollment/join state, compliance (with the specific non-compliant policies named), assigned configuration profiles, and app install status from Microsoft Graph. Read-only: it never changes the tenant. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DESKSUP-Validation.ps1 -DeviceName device-name-or-serial

How to run: powershell -ExecutionPolicy Bypass -File .\DESKSUP-Validation.ps1 -DeviceName device-name-or-serial